

智慧银行实验教程

实验二：环境搭建与 AI 协作

姓名 学号 班级

2026 年 6 月 25 日

1 实验目的

1. 掌握 AI 辅助开发环境（IDE）的安装与基本操作；
2. 完成 Python、Node.js、Git、.NET SDK 等核心运行时的安装与版本验证；
3. 安装 uv 包管理器并理解其在 Python 生态中的作用；
4. 学习环境变量配置方法，为 MCP 服务运行做好准备；
5. 掌握 Git 分布式版本控制的基本操作（clone、add、commit、push）；
6. 学习 CNB 代码托管平台的认证机制（Personal Access Token），理解权限管理的重要性；
7. 了解 C 盘空间管理，掌握将开发工具缓存迁移至非系统盘的方法；
8. 通过贪吃蛇游戏项目的完整开发与部署流程，实践 AI 协作编程的全生命周期。

2 实验环境

表 1: 实验环境一览

类别	项目	版本/配置
操作系统	Windows 10/11 (x64)	—
AI IDE	WorkBuddy / Qoder	最新版本
Python	托管版本	3.13.12
Node.js	托管版本	v22.22.2
npm / npx	随 Node.js 安装	10.9.7
uv	pip 安装	0.11.24
.NET SDK	winget + dotnet-install 脚本	10.0.301
Git	系统预装	—
Stata	本地安装	StataMP-64.exe (E 盘)

3 实验步骤

3.1 步骤 1: 检查系统基础环境

首先验证操作系统已具备的基础软件版本，确保满足课程最低要求。

```
# 检查 Python 版本 (要求 >= 3.10)
python --version
# 输出: Python 3.13.12

# 检查 Node.js 版本 (要求 >= v20)
node --version
# 输出: v22.22.2

# 检查 npm/npx
npm --version
npx --version
# 输出: 10.9.7

# 检查 Git
git --version
```

验证结果: Python 3.13.12 $\geq$ 3.10, Node v22.22.2 $\geq$ v20, 全部满足课程要求。

3.2 步骤 2：安装 uv 包管理器

uv 是本课程 MCP 服务的核心依赖管理工具，多个 MCP 服务器（fetch、word、ppt、stata-mcp）通过 uvx 命令启动。

```
pip install uv
# 验证安装
uv --version
# 输出: uv 0.11.24

uvx --version
# 输出: uvx 0.11.24
```

3.3 步骤 3：安装 .NET SDK 与 WindowsMCP.Net

WindowsMCP.Net 是本课程桌面自动化 MCP 服务的关键组件。初次安装 .NET 9.0.315 后发现该工具要求 .NET 10.0 运行时，因此升级到 .NET 10.0.301。

```
# 安装 .NET 9.0（初始尝试）
winget install Microsoft.DotNet.SDK.9 --accept-package-agreements

# 发现 WindowsMCP.Net 需要 .NET 10，继续安装
winget install Microsoft.DotNet.SDK.10 --accept-package-agreements

# 验证
dotnet --version
# 输出: 10.0.301

# 安装 WindowsMCP.Net dotnet 工具
dotnet tool install --global WindowsMCP.Net
# 输出: 已成功安装工具 "windowsmcp.net" (版本 "0.2.6")
```

3.4 步骤 4：配置环境变量

为支持 MCP 服务器和开发工具链，设置以下用户级环境变量：

表 2: 环境变量配置清单

变量名	值	用途
STATA_PATH	E:\Stata18\StataMP-64.exe	stata-mcp 定位 Stata
EXCEL_MCP_PAGING_CELLS_LIMIT	10000	Excel MCP 分页限制
MCP_STATA_LOGLEVEL	INFO	Stata MCP 日志级别
DOTNET_ROOT	E:\dotnet	.NET 安装根目录
UV_CACHE_DIR	E:\智慧银行实验作业\uv-cache	uv 缓存路径
PIP_CACHE_DIR	E:\智慧银行实验作业\pip-cache	pip 缓存路径

3.5 步骤 5: C 盘空间优化——缓存迁移

由于开发工具链的缓存文件占用大量 C 盘空间（约 1.18 GB），决定将所有缓存迁移至 E 盘课程目录下。

```
# 创建目标目录
mkdir "E:\智慧银行实验作业\npm-cache"
mkdir "E:\智慧银行实验作业\uv-cache"
mkdir "E:\智慧银行实验作业\pip-cache"
mkdir "E:\智慧银行实验作业\dotnet-tools"

# 迁移 npm 缓存
cp -r "%LOCALAPPDATA%\npm-cache\*" "E:\智慧银行实验作业\npm-cache\"
rm -rf "%LOCALAPPDATA%\npm-cache"
npm config set cache "E:\智慧银行实验作业\npm-cache"

# 迁移 uv 缓存
cp -r "%LOCALAPPDATA%\uv\cache\*" "E:\智慧银行实验作业\uv-cache\"
rm -rf "%LOCALAPPDATA%\uv\cache"

# 迁移 pip 缓存
cp -r "%LOCALAPPDATA%\pip\cache\*" "E:\智慧银行实验作业\pip-cache\"
rm -rf "%LOCALAPPDATA%\pip\cache"

# 迁移 .NET 工具
cp -r "%USERPROFILE%\dotnet\tools\*" "E:\智慧银行实验作业\dotnet-tools\"
```

3.6 步骤 6: .NET SDK 跨盘重装

原.NET SDK (9.0 + 10.0) 安装在 C:\Program Files\dotnet, 占用约 1.49 GB。通过卸载后使用 dotnet-install.ps1 脚本重装至 E 盘。

```
# 卸载原安装
winget uninstall Microsoft.DotNet.SDK.9 --silent
winget uninstall Microsoft.DotNet.SDK.10 --silent

# 使用官方脚本安装到 E 盘
Invoke-WebRequest -Uri "https://dot.net/v1/dotnet-install.ps1" -OutFile dotnet-
install.ps1
./dotnet-install.ps1 -Channel 10.0 -InstallDir "E:\dotnet"
# 输出: dotnet 10.0.301 installed to E:\dotnet
```

3.7 步骤 7: Git 仓库建立与 CNB 版本控制

Git 是本课程代码管理的核心工具。CNB (cnb.cool) 是课程指定的代码托管平台, 提供与 GitHub 兼容的 Git 操作体验, 但使用 Personal Access Token (PAT) 进行认证。

3.7.1 7.1 创建 CNB 仓库

在 CNB 平台上创建课程专属仓库 sichuanagr202301729/zhihuiyinhang, 用于存放所有实验项目的源代码。

3.7.2 7.2 克隆仓库

```
# 使用 Token 认证方式克隆仓库
git clone https://cnb:<TOKEN>@cnb.cool/sichuanagr202301729/zhihuiyinhang.git
```

3.7.3 7.3 配置 Git 用户信息

```
git config user.name "cnb"
git config user.email "cnb@cnb.cool"
```

3.7.4 7.4 CNB 认证机制——Personal Access Token

CNB 平台使用访问令牌 (PAT) 进行认证, 而非传统的用户名 + 密码方式。令牌的权限范围可精细控制:

表 3: CNB Token 权限说明

权限标识	含义	用途
repo-code:r	仓库代码读取	克隆、拉取代码
repo-code:rw	仓库代码读写	推送代码、创建分支
repo-code:admin	仓库代码管理	修改仓库设置、删除分支

3.8 步骤 8: AI 协作实战——贪吃蛇游戏开发

3.8.1 8.1 项目背景

为了实践 AI 协作编程的全流程，选择开发一个经典的贪吃蛇网页游戏作为实验项目。该游戏使用纯 HTML + CSS + JavaScript 实现，无需任何外部依赖，可直接在浏览器中运行。

3.8.2 8.2 功能设计

- **核心玩法：**使用方向键控制蛇的移动，吃到食物得分并增长蛇身；
- **碰撞检测：**蛇头碰到墙壁或自身时游戏结束；
- **分数系统：**实时显示当前得分和最高分；
- **游戏控制：**开始、暂停、重新开始功能；
- **响应式设计：**适配不同屏幕尺寸。

3.8.3 8.3 AI 协作过程

整个贪吃蛇游戏由 AI 助手（WorkBuddy）生成完整的 HTML 文件，包括游戏画布、蛇的移动逻辑、食物生成、分数统计和样式美化。开发者主要负责提出需求和验证功能，代码生成由 AI 完成。这充分体现了“低代码、高集成”的课程理念。

3.8.4 8.4 上传到 CNB 仓库

```
# 将游戏文件复制到本地仓库
cp "E:/智慧银行实验作业/index.html" /tmp/zhihuuiyinhang/snake_game.html

# 添加、提交并推送
cd /tmp/zhihuuiyinhang
git add snake_game.html
git commit -m "添加贪吃蛇游戏项目 (snake game)"
```

```
git push origin master
```

4 实验结果

4.1 环境验收清单

按照课本附录 A 的环境检查清单逐项验证：

表 4: 环境验收清单

序号	检查项	状态
1	Python $\geq$ 3.10	✓ 3.13.12
2	Node.js $\geq$ v20	✓ v22.22.2
3	npm / npx 可用	✓ 10.9.7
4	uv / uvx 已安装	✓ 0.11.24
5	.NET SDK $\geq$ 10.0	✓ 10.0.301
6	Git 可用	✓
7	STATA_PATH 已配置	✓ E:\Stata18
8	EXCEL 变量已配置	✓ 10000
9	Stata 可执行文件存在	✓
10	npm 缓存已迁移	✓ E 盘
11	uv 缓存已迁移	✓ E 盘
12	pip 缓存已迁移	✓ E 盘
13	.NET 工具已迁移	✓ E 盘
14	.NET SDK 已迁移	✓ E 盘
15	环境变量已持久化	✓ 用户级

4.2 C 盘空间释放情况

表 5: C 盘空间释放汇总

组件	大小 (MB)	新位置
npm 缓存	214.0	E:\智慧银行实验作业\npm-cache
uv 缓存	586.9	E:\智慧银行实验作业\uv-cache
pip 缓存	381.4	E:\智慧银行实验作业\pip-cache
.NET 工具	0.2	E:\智慧银行实验作业\dotnet-tools
.NET SDK	1,486.3	E:\dotnet
合计	2,668.8	—

共从 C 盘释放约 **2.67 GB** 空间。

4.3 贪吃蛇游戏项目验收

表 6: 贪吃蛇游戏功能验收

功能	实现方式	状态
蛇的移动	JavaScript Canvas + 键盘事件	✓
食物随机生成	Math.random() 坐标计算	✓
碰撞检测	边界判定 + 自身碰撞判断	✓
分数系统	实时计分 + LocalStorage 最高分	✓
游戏控制	开始/暂停/重开按钮	✓
响应式布局	CSS flexbox + Canvas 自适应	✓
CNB 版本管理	Git commit + push to master	✓

4.4 Git 仓库验证

表 7: CNB 仓库交付物

文件	说明
snake_game.html	贪吃蛇游戏（单文件，约 500 行）
ch02-...tex / ch03-...tex	LaTeX 实验报告源文件

仓库地址: <https://cnb.cool/sichuanagr202301729/zhihuiyinhang> (master 分支)。

5 问题与解决

5.1 问题 1: pip 安装 uv 速度慢

现象: 默认 PyPI 源下载速度极慢 (<100 KB/s)。

解决: 使用清华镜像源加速。

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

5.2 问题 2: .NET 9 不兼容 WindowsMCP.Net

现象: 安装 WindowsMCP.Net 时提示需要 .NET 10.0 运行时, 而当时系统只有 .NET 9.0.315。

解决: 查阅 NuGet 官方页面确认版本依赖, 通过 winget 安装 .NET SDK 10.0.301。安装过程中注意 .NET 10 安装包约 203 MB, 需耐心等待下载完成。

5.3 问题 3: npm 缓存目录清理不彻底

现象: 使用 `cp -r` 迁移后执行 `rm -rf` 删除原目录, 但 PowerShell 检查发现目录仍存在。

解决: 部分文件可能被进程占用, 重新执行一次 `rm -rf` 成功清理。建议迁移前关闭所有使用 npm 的终端窗口。

5.4 问题 4: 环境变量不立即生效

现象: 通过 `[Environment]::SetEnvironmentVariable` 设置的环境变量在当前终端中不生效。

解决: 用户级环境变量需要重启终端窗口才能加载。此问题不影响配置正确性, 重启后所有变量正常读取。

5.5 问题 5: CNB Token 权限不足导致推送失败

现象: 创建令牌名 “123” 后执行 `git push`, 报 “Permission denied” — 远程服务器拒绝写入操作。

原因分析: 初次创建的 Token 仅具备 `repo-code:r` (只读) 权限, 只能执行 `clone/pull` 操作, 无法 `push`。

解决: 在 CNB 平台重新创建令牌 (令牌名 “1234”), 勾选 `repo-code:rw` (读写) 权限。使用新令牌更新 Git 远程地址后成功推送。

```
# 使用新令牌更新远程地址
git remote set-url origin https://cnb:<NEW_TOKEN>@cnb.cool/...
```

```
git push origin master # 成功
```

启示：这是典型的权限管理实践案例。在金融业务场景中，API 密钥和访问令牌的权限范围需要遵循“最小权限原则”（Principle of Least Privilege），只授予完成当前任务所需的最小权限。

6 实验心得

通过本次实验，我深刻体会到 AI 辅助开发环境搭建的重要性与复杂性：

收获方面：

1. 掌握了多种包管理工具（pip、npm、winget、dotnet）的使用方法，了解了不同生态的工具链差异；
2. 学会了通过环境变量管理系统配置，理解了用户级与系统级变量的区别；
3. 认识到 C 盘空间管理是开发环境维护的重要一环，学会了缓存的跨盘迁移方法；
4. 体验了从版本不兼容到成功安装的完整排障过程，提升了独立解决问题的能力。

Git 版本控制与 CNB 协作体会：

1. 掌握了 CNB 平台基于 Token 的认证机制，理解了 `repo-code:r` 与 `repo-code:rw` 的权限差异，这与金融业务中 API 密钥的权限分级管理理念一脉相承；
2. 完成了从 clone、add、commit 到 push 的完整 Git 工作流，理解了版本控制在团队协作中的核心价值；
3. Token 权限问题的排查过程让我认识到：看似简单的“推送失败”背后可能涉及认证、权限、网络等多层原因，需要系统性地逐层排查。

贪吃蛇游戏项目反思：

1. 这是一个典型的 AI 协作编程案例——从需求描述到代码生成，AI 助手完成了约 95% 的编码工作，开发者主要负责需求定义和功能验证；
2. 单文件 HTML 项目的部署极为简洁（无需构建工具），但这也意味着代码规模扩大后维护困难。后续第 10 章 CRM 项目的组件化架构将是更好的工程实践；
3. 游戏开发过程中，碰撞检测逻辑（蛇头与墙壁、蛇头与自身）是算法设计的核心，这让我联想到金融风控中的“边界条件检测”——同样是确保系统不会越过安全边界。

不足与改进：

1. 初期未了解 WindowsMCP.Net 对.NET 版本的精确要求，导致安装了不兼容的版本，浪费了时间。今后安装工具前应先查阅官方文档确认依赖版本；
2. 环境变量配置可以在所有安装完成后再统一设置，而不是分散操作，这样更便于管理和复查；
3. LaTeX 排版环境尚未安装（MiKTeX），后续实验中需要补充配置；
4. 贪吃蛇项目缺乏单元测试和持续集成（CI），这在第 9–12 章的 BMAD 方法论中将得到补充。

AI 协作体会：本次实验中，AI 助手（WorkBuddy）在命令生成、版本验证、错误排查、游戏代码生成等方面提供了极大帮助。特别是.NET 版本不兼容问题的定位、缓存迁移方案的设计、以及贪吃蛇游戏从零到一的完整开发，AI 均能快速给出准确的解决方案。这让我切实体会到“AI 增强型金融人才”中 AI 工具的价值——不是替代人类思考，而是将开发者从繁琐的配置和基础编码中解放出来，聚焦于业务逻辑和架构决策。